

it can be many characters that together comprise the token.

As the `StringTokenizer` class object moves its way through a `String`, it automatically knows where it is in the `String`, you do not have to keep track of that. When you instantiate an object of type `StringTokenizer`, you have three alternatives arguments for the Constructor

```
1.) String stringtobetokenized
2.) String stringtobetokenized,
   String delimiter
3.) String stringtobetokenized,
   String delimiter,
   boolean returnTokens
Default delimiters: " \n, \t, \r "
Default returnTokens = false.
```

6.6.1 StringTokenizer Class: Methods

```
int          countTokens ()
boolean      hasMoreTokens ()
boolean      hasMoreElements ()
Object       nextElement ()
String       nextToken ()
String       nextToken( String delimiter )
```

`countTokens()` method counts how many more times this tokenizer object's `nextToken` can be counted.

```
int countTokens ()
```

It returns an integer with that number.

`hasMoreTokens()` simply tests if there are more tokens available from this tokenizer's `String`.

`boolean hasMoreTokens()` method if returns true, it anticipates that the a call to method `nextToken()` right after this would succeed in returning a token.

`hasMoreElements()` is used to override a method inherited from the `Enumeration` interface, which this class implements.

```
boolean hasMoreElements ()
```

In effect, this does the same thing as the `hasMoreTokens()` method. true if there are more tokens.

`nextToken()` method is the workhorse. It gets the next token waiting in the `String` that was used to instantiate this object.